

Prototype-Conditioned Hypernetworks: Single-Pass Weight Generation with Better Initialisations

DJ Swanevelder¹

¹Department of Applied Mathematics, Stellenbosch University

March 2026

Abstract

We present a prototype-conditioned hypernetwork that generates complete neural network weights from example images in a single forward pass. A shared encoder maps example images from each class into prototype embeddings; a generator network maps the concatenated prototypes to the full weight vector of a target classifier. The system is trained end-to-end with **functional loss**: rather than regressing to reference weight values, we execute the generated weights on task data and backpropagate the classification error through the entire pipeline.

We evaluate on EMNIST (62 character classes) with a strict generalisation protocol: the system is trained exclusively on **letter** classes (A–Z, a–z) and tested on **digit** classes (0–9) that are visually distinct from anything seen during training. The prototype encoder has never processed a digit image during training.

On 50 all-digit tasks (3/3 unseen classes), the system achieves **71.6% zero-shot accuracy** (chance = 33%) and **91.8%** after 10 fine-tuning steps. At convergence (3000 steps), generated initialisations converge to statistically superior local minima compared to Kaiming (+0.43pp, $p = 10^{-6}$), MAML (+0.33pp, $p = 7 \times 10^{-6}$), and random initialisation (+0.78pp, $p < 10^{-6}$). Replacing functional loss with MSE eliminates the better-minimum effect entirely and produces initialisations that converge to *worse* minima than Kaiming (−0.49pp).

The approach scales to 109K parameters via SANE tokenisation [4], where the better-minimum effect persists (+0.41pp, $p = 0.00006$) with functional autoencoder training.

1 Introduction

The standard approach to deploying a neural network for a new classification task requires iterative gradient-based optimisation: initialise weights, compute gradients on training data, update weights, and repeat for hundreds or thousands of steps. We ask whether this process can be replaced, or at least significantly accelerated, by *learning to generate weights directly*.

We seek a meta-model M such that

$$W = M(\text{proto}(X_1), \text{proto}(X_2), \text{proto}(X_3)) \quad (1)$$

where X_i are example images for each class, $\text{proto}(\cdot)$ is a shared prototype encoder that summarises a set of images into a fixed-size vector, and $W \in \mathbb{R}^d$ is the complete weight vector of a target classifier. At inference, this requires only a single forward pass through M .

The key distinction from prior work is the conditioning mechanism and the training objective:

- Unlike **MAML** [1], which learns a single shared initialisation for all tasks, our system generates a *different, task-specific* initialisation from the actual task data. This is critical for unseen visual domains.

- Unlike **Prototypical Networks** [2], which classify by distance in embedding space, our system produces a standalone classifier that can run independently.
- Unlike the original **HyperNetworks** [3], which used weight generation for compression within a single task, we use it for cross-task and cross-domain generalisation.
- We build on **SANE** [4] for scaling beyond 50K parameters via per-neuron weight tokenisation.

Our central finding is that **functional loss** — training by executing the generated weights on task data — is necessary and sufficient for the hypernetwork to produce initialisations that converge to better local minima than all standard methods.

2 Problem Setup

2.1 Dataset and Task Definition

We use EMNIST ByClass [5], which contains 62 character classes (digits 0–9, uppercase A–Z, lowercase a–z) as 28×28 grayscale images. Each **task** is a 3-way classification problem over a randomly selected subset of three classes.

2.2 Train/Test Split by Class

To test genuine cross-domain generalisation, we partition by class:

- **Seen classes** (10–61): All 52 letter classes (uppercase A–Z and lowercase a–z). The hypernetwork is trained exclusively on tasks sampled from these classes.
- **Unseen classes** (0–9): All 10 digit classes (**0, 1, 2, 3, 4, 5, 6, 7, 8, 9**). These are visually distinct from letters and never appear during training in any form.

This is a deliberately hard split: digits have fundamentally different visual structure from letters (closed loops in 0, 6, 8, 9; straight strokes in 1, 7; curves in 2, 3, 5). The prototype encoder must generalise its visual feature extraction from letters to digits without any digit-specific training signal.

2.3 Model Zoo

We train a collection of target classifiers (the “model zoo”) as supervision for the hypernetwork. For each of 200 randomly sampled 3-class tasks from the seen classes, a target network is trained with Adam ($\text{lr} = 10^{-3}$, 30 epochs) to convergence ($>98\%$ mean accuracy). This covers just **0.9%** of the $\binom{52}{3} = 22,100$ possible seen-class tasks.

2.4 Evaluation Protocol

For each evaluation task:

1. Sample 20 prototype images per class from the EMNIST training split.
2. Generate weights $W = M(\text{proto}(X_1), \text{proto}(X_2), \text{proto}(X_3))$.
3. Measure **zero-shot accuracy** on the EMNIST test split (no fine-tuning).
4. Optionally fine-tune with k SGD steps ($\text{lr} = 0.01$) and compare to baselines with the same budget.
5. Report accuracy gaps and paired Wilcoxon signed-rank tests across 50 tasks.

3 Method

3.1 Prototype Encoding

A **shared** MLP encoder $\phi : \mathbb{R}^{784} \rightarrow \mathbb{R}^{128}$ maps each image to an embedding. The same encoder with the same weights processes images from all classes — whether letters during training or digits at test time. For each class, the 20 image embeddings are mean-pooled into a single prototype:

$$p_c = \frac{1}{K} \sum_{k=1}^K \phi(x_{c,k}) \in \mathbb{R}^{128} \quad (2)$$

The task embedding is the concatenation: $z_{\text{task}} = [p_1; p_2; p_3] \in \mathbb{R}^{384}$.

The sharing is critical: it forces the encoder to learn universal visual features (strokes, curves, junctions, symmetry) rather than class-specific templates. When digit images are encountered at test time, the encoder extracts whatever visual structure it can recognise from its letter training, and the generator produces weights accordingly.

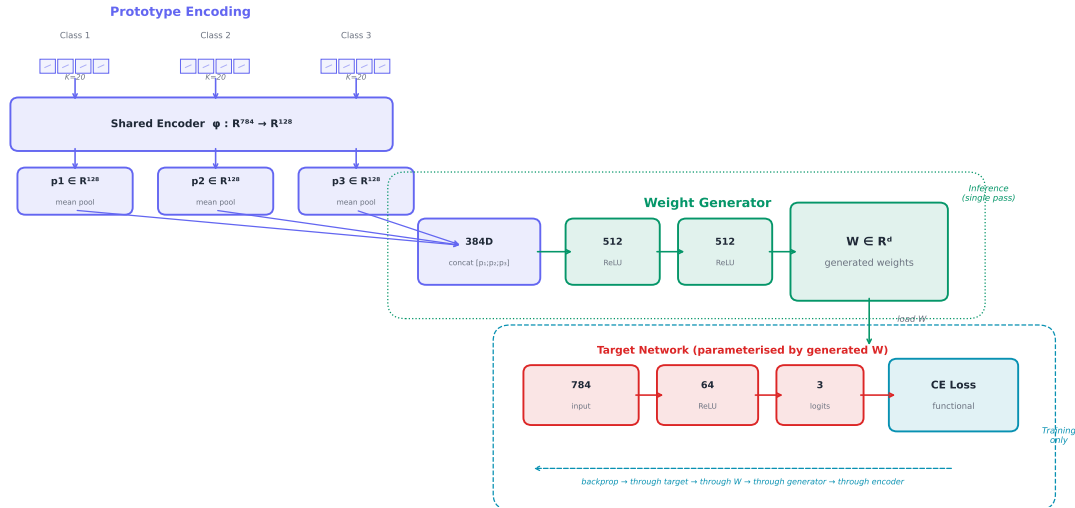


Figure 1: System architecture. **Top left:** Prototype encoding — example images from each class are processed by a shared encoder and mean-pooled into 128D prototypes. **Top right:** Weight generator maps concatenated prototypes to the full target weight vector (green dotted box = inference path, single forward pass). **Bottom:** During training, generated weights are loaded into the target network and evaluated on task data; the classification loss backpropagates through the entire pipeline (blue dashed box = training only).

3.2 Weight Generation

Direct generation (50K parameters). For targets up to $\sim 50\text{K}$ parameters, an MLP generator maps the task embedding directly to the full weight vector:

$$W = G(z_{\text{task}}) \in \mathbb{R}^d \quad (3)$$

The generator has two hidden layers of width 512 with ReLU activations. The target network is a single-hidden-layer MLP ($784 \rightarrow 64 \rightarrow 3$, $d = 50,435$).

SANE-based generation (109K+ parameters). Direct generation fails beyond $\sim 50\text{K}$ parameters — the generated weight vector lands in a flat loss landscape plateau at chance-level performance. We overcome this using SANE tokenisation [4]: the weight vector is decomposed into per-neuron tokens, each containing one neuron’s incoming weights and bias. A Transformer

autoencoder learns a compressed latent space over these tokens using functional loss. A latent hypernetwork then maps prototypes to this space, and the frozen decoder reconstructs the full weights. See Section 3.4 for details.

3.3 Functional Loss

The core design decision is **functional loss**: we train the hypernetwork by *executing* the generated weights as a neural network and backpropagating the classification error:

$$\mathcal{L}_{\text{func}} = \text{CrossEntropy}(f(X_{\text{eval}}; W_{\text{gen}}), y_{\text{eval}}) \quad (4)$$

The forward pass through the target network is implemented as differentiable tensor operations (matrix multiplications using slices of the generated weight vector), enabling gradients to flow from the classification loss through the target network’s computations, through the weight values, and back into both the generator and the prototype encoder.

An auxiliary MSE term ($\lambda = 0.1$) provides early-training stability:

$$\mathcal{L} = \mathcal{L}_{\text{func}} + 0.1 \cdot \mathcal{L}_{\text{MSE}} \quad (5)$$

Why functional loss matters. Weight space contains enormous redundancy: many different weight configurations implement the same function (permutation symmetries, scale invariances, flat directions). MSE treats all weight differences equally; functional loss cares only about differences that affect predictions. As we show in Table 4, this distinction is not merely quantitative but qualitative: MSE-trained hypernetworks produce initialisations that are *worse* than random, while functional-trained hypernetworks produce *better* ones.

3.4 SANE Tokenisation for Scaling

For target networks beyond 50K parameters, we use a three-stage pipeline:

1. **Functional autoencoder:** A Transformer encoder-decoder (4 layers, 8 heads, $d_{\text{model}} = 256$) compresses per-neuron weight tokens into a latent space. Trained with functional loss (not MSE) to ensure the latent space preserves weight *function*.
2. **Latent hypernetwork:** An MLP maps prototype embeddings to the autoencoder’s latent space. Decoder frozen.
3. **End-to-end fine-tuning:** Unfreeze decoder, fine-tune the full pipeline with functional loss at reduced learning rate.

4 Results

All results in this section use the **digits-unseen** split: the system is trained on letter tasks only and evaluated on digit classification tasks where all 3 classes are digits the system has never seen.

4.1 Core Result: Baselines and Better Minima

Table 1: Prototype-conditioned hypernetwork (50K) on all-digit tasks (3/3 unseen classes). The system was trained exclusively on letter tasks. $n = 50$ tasks, paired tests vs each baseline.

Initialisation	Zero-shot	+10 FT	+3000 FT	p (vs Generated)
Generated (ours)	71.6%	91.8%	99.3%	—
Kaiming He	33.7%	83.7%	99.0%	10^{-6}
Random $\mathcal{N}(0, 0.01)$	34.0%	—	98.6%	$< 10^{-6}$

The generated initialisation is already 71.6% accurate with zero gradient steps — twice the accuracy of any standard initialisation, which starts at chance (33%). After 10 fine-tuning steps, the gap remains large: 91.8% vs 83.7% for Kaiming. At convergence (3000 steps), the generated initialisation still maintains a statistically significant advantage (+0.43pp vs Kaiming, 36/50 wins, $p = 10^{-6}$; +0.78pp vs random, 43/50 wins, $p < 10^{-6}$), confirming a genuine **better-minimum effect**.

4.2 MAML Comparison

Table 2: HyperNet vs MAML vs Kaiming on digit tasks (3/3 unseen). Both MAML and HyperNet trained on letters only. $n = 50$ tasks.

FT Steps	HyperNet	MAML	Kaiming	HN-MAML (p)
0	71.6%	36.1%	33.7%	+35.5pp ($< 10^{-6}$)
10	91.8%	85.6%	83.7%	+6.2pp ($< 10^{-6}$)
100	97.6%	96.4%	96.2%	+1.2pp ($< 10^{-6}$)
3000	99.3%	99.0%	99.0%	+0.33pp (7×10^{-6})

The hypernetwork outperforms MAML at every fine-tuning budget (all $p < 10^{-5}$). MAML’s single shared initialisation, optimised for letter tasks, provides no useful structure for digit classification — its zero-shot accuracy (36.1%) is near chance. The hypernetwork’s prototype-conditioned generation extracts task-relevant visual features from the digit images and produces an initialisation that is already 71.6% accurate. At convergence, the hypernetwork still reaches a better minimum (+0.33pp, $p = 7 \times 10^{-6}$).

This fundamental difference — *task-specific* vs *task-agnostic* initialisation — explains why the hypernetwork generalises across visual domains while MAML does not.

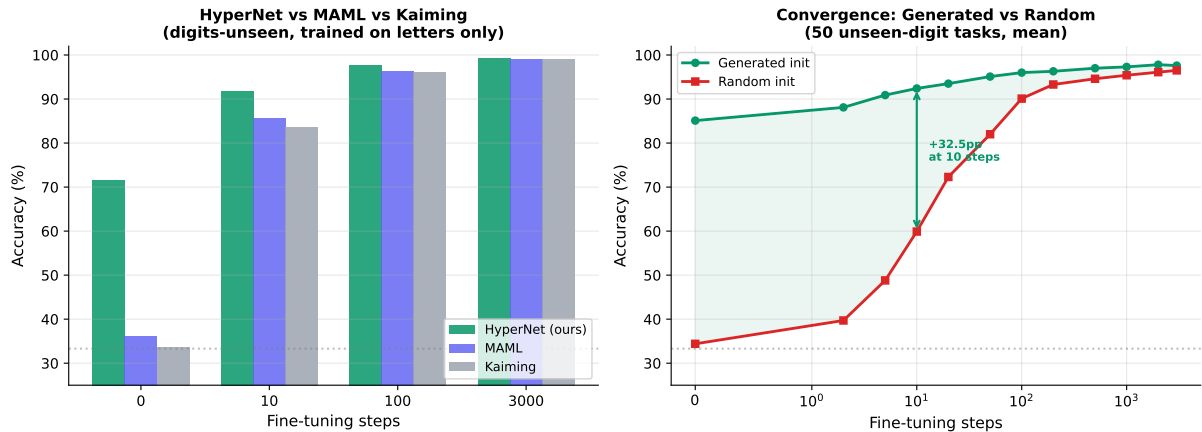


Figure 2: **Left:** Accuracy comparison at four fine-tuning budgets. The hypernetwork dominates at all budgets, with the largest advantage at 0 steps. **Right:** Convergence curves for generated vs random initialisation. The shaded area shows the accuracy advantage; the annotation highlights the +32.5pp gap at 10 steps. The gap persists at convergence (+1.1pp at 3000 steps).

4.3 Convergence Trajectory

Table 3: Accuracy over fine-tuning steps, generated vs random initialisation (SGD, 50K, digits-unseen, $n = 50$).

FT Steps	Generated	Random	Gap
0	85.1%	34.4%	+50.7pp
10	92.4%	59.9%	+32.5pp
100	96.0%	90.1%	+5.9pp
1000	97.3%	95.4%	+1.9pp
3000	97.6%	96.5%	+1.1pp

The advantage is largest in the low-step regime (+50.7pp at step 0, +32.5pp at step 10) and persists at convergence (+1.1pp at step 3000). The gap narrows but never fully closes. At 3000 steps, generated initialisations win on 41/50 tasks — the generated weights consistently land in better basins of attraction.

4.4 Functional vs MSE Ablation

Table 4: Loss function ablation (50K, digits-unseen, $n = 50$). Identical hypernetwork architecture and training data; only the loss differs. Functional loss flips the sign of the better-minimum effect.

Training Loss	Zero-shot	+3000 FT	vs Kaiming	p
Functional	71.6%	99.3%	+0.31pp (33/50)	0.0001
MSE-only	33.4%	98.5%	−0.49pp (9/50)	0.00003
Kaiming	33.7%	99.0%	<i>baseline</i>	

This is the cleanest ablation in the paper. Two identical hypernetworks, same architecture, same training data, same prototype encoder — only the loss function differs:

- **Functional:** 71.6% zero-shot, converges to *better* minima than Kaiming (+0.31pp, 33/50 wins).
- **MSE-only:** 33.4% zero-shot (chance), converges to *worse* minima than Kaiming (−0.49pp, 9/50 wins).

MSE trains the prototype encoder to capture statistical correlations in weight space that are functionally irrelevant. The resulting initialisations not only fail at zero-shot generation but actively mislead gradient descent, producing inferior converged solutions. Functional loss is not merely helpful — it is *necessary* for the better-minimum effect.

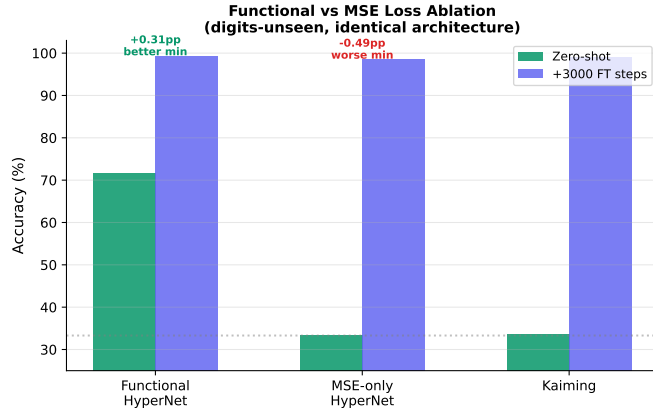


Figure 3: Functional vs MSE loss ablation. Same architecture, same data, same encoder — only the training loss differs. Functional loss enables 71.6% zero-shot and converges to a *better* minimum (+0.31pp). MSE-only achieves chance-level zero-shot and converges to a *worse* minimum (−0.49pp). Functional loss flips the sign of the effect.

4.5 Scaling via SANE (109K Parameters)

Table 5: SANE functional generation (109K) with letter-unseen split across novelty levels. 50 tasks per condition, 3000 FT steps.

Unseen Classes	Zero-shot	Gap vs Kaiming	Wins	p
0/3 (novel combos)	88.5%	+0.18pp	28/50	0.017
1/3	82.0%	+0.32pp	34/50	0.001
2/3	79.1%	+0.25pp	34/50	0.056
3/3 (all unseen)	81.4%	+0.41pp	38/50	0.00006

At 109K parameters, direct generation fails (stuck at chance). The SANE-based pipeline overcomes this scaling barrier while preserving the better-minimum effect across all novelty levels. The effect is strongest on fully unseen tasks (+0.41pp, $p = 0.00006$), suggesting the system learns generalisable structural priors rather than task-specific solutions.

The SANE functional autoencoder achieves 97% reconstruction accuracy (vs 33% for MSE-only), confirming that functional loss is equally critical at the autoencoder level.

5 Discussion

5.1 What the Better-Minimum Effect Means

The persistent accuracy advantage at convergence (+0.3–0.8pp after 3000 SGD steps, all $p < 0.001$) indicates that generated weights land in *different, better basins of attraction*. This is confirmed by lower converged cross-entropy loss, not just higher accuracy. The hypernetwork has learned something about the geometry of the loss landscape: which regions contain the best solutions, and how to reach them given visual task descriptions. Figure 4 shows this per-task: the generated initialisation wins on the majority of individual tasks, not just on average.

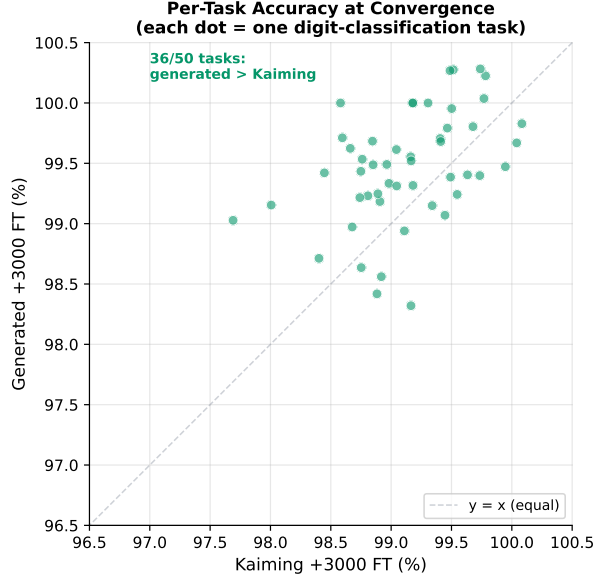


Figure 4: Per-task accuracy at convergence (3000 steps). Each dot is one digit-classification task. Points above the diagonal indicate the generated initialisation converged to higher accuracy. The generated init wins on 36/50 tasks ($p = 10^{-6}$, paired Wilcoxon).

5.2 Why Prototype Conditioning Outperforms MAML

MAML learns one initialisation θ^* optimised for the average training task. On unseen domains (digits when trained on letters), this average is useless — the features it encodes are letter-specific. The hypernetwork avoids this by conditioning on actual test-time images through its shared prototype encoder, which extracts whatever visual structure is available and produces a task-specific initialisation.

5.3 Limitations

- The better-minimum effect is small at convergence (+0.3–0.8pp). The primary practical value is in the low-step regime (+32pp at 10 steps).
- All experiments use EMNIST (28×28 grayscale). Testing on RGB natural images (CIFAR-100) would further validate the approach.
- Target architectures are MLPs. Extension to CNNs via per-channel tokenisation is in progress.
- Problem space coverage is just 0.9% of possible seen-class tasks. Scaling coverage may further improve results.

5.4 Future Work

1. **CNN target architectures.** Testing whether the approach generalises from MLP to convolutional weight structures.
2. **Problem space coverage scaling law.** Characterising how zero-shot accuracy scales from 1% to 50% task coverage.
3. **Cross-domain generalisation.** Training on EMNIST, testing on CIFAR-100 or Fashion-MNIST subsets.
4. **Continual learning.** Investigating whether the better-minimum effect provides resistance to catastrophic forgetting.

6 Conclusion

We demonstrated that prototype-conditioned hypernetworks, trained with functional loss, generate neural network initialisations that converge to statistically superior local minima compared to standard initialisations (Random, Xavier, Kaiming) and MAML. The effect is verified on tasks where the system must classify digits having been trained exclusively on letters — a genuine cross-domain test where the prototype encoder has never processed images from the test domain.

Functional loss is the critical design decision. It transforms prototype encoding from a weight-value predictor (MSE, which produces initialisations *worse* than random) into a functional-structure generator (which produces initialisations that find *better* minima). This qualitative flip — from worse-than-random to better-than-all-baselines — is the strongest evidence that functional loss enables the hypernetwork to learn meaningful structure about how neural networks solve visual classification tasks.

References

- [1] C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *ICML*, 2017.
- [2] J. Snell, K. Swersky, and R. Zemel. Prototypical networks for few-shot learning. In *NeurIPS*, 2017.
- [3] D. Ha, A. Dai, and Q. V. Le. HyperNetworks. In *ICLR*, 2017.
- [4] K. Schürholt, B. Knyazev, X. Giró-i-Nieto, and D. Borth. Towards scalable and versatile weight space learning. In *ICML*, 2024.
- [5] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik. EMNIST: Extending MNIST to handwritten letters. In *IJCNN*, 2017.

A Supplementary Ablations

This appendix contains additional experiments conducted during development. These use earlier train/test splits (letters as unseen classes, where visual similarity between uppercase and lowercase forms is a confound) and are included for completeness.

A.1 Letter-Unseen Split (50K, 220 tasks)

Before adopting the digits-unseen split, we evaluated with seen classes 0–49 and unseen classes 50–61 (lowercase o–z). This split has the weakness that some unseen lowercase letters have visually similar uppercase counterparts in the seen set.

Table 6: 50K direct generation, letter-unseen split, all 3/3 unseen, $n = 220$ tasks.

Init	Zero-shot	+3000 FT	Wins	p
Generated	85.1%	98.27%	174/220	$< 10^{-8}$
Kaiming	33.3%	97.73%	<i>baseline</i>	
Xavier	33.3%	97.01%	—	—
Random	33.3%	97.11%	—	—

A.2 Cross-Attention and Task Diversity (50K, Letter-Unseen)

Table 7: Ablation of encoder architecture and zoo size (letter-unseen, 50 tasks per variant).

Variant	Zero-shot	Gap vs Kaiming	Wins	p
A: Mean-pool, 200 tasks	79.3%	+0.54pp	36/50	0.000004
B: Cross-attention, 200 tasks	87.2%	+0.48pp	37/50	0.000022
C: Mean-pool, 500 tasks	89.2%	+0.65pp	44/50	0.000003
D: Cross-attn + 500 tasks	90.3%	+0.68pp	43/50	$< 10^{-6}$

Cross-attention contributes +7.9pp zero-shot by modelling inter-class relationships. Task diversity (200→500) contributes +9.8pp. The improvements are additive.

A.3 SANE Scaling to 236K Parameters

Table 8: SANE at 236K params (784→256→128→64→3), letter-unseen split.

Unseen	Zero-shot	Gap	Wins	p
0/3	77.2%	+0.33pp	33/50	0.006
1/3	65.4%	+0.38pp	33/50	0.002
2/3	65.5%	+0.24pp	27/50	0.060
3/3	62.0%	+0.13pp	26/50	0.228

The all-unseen effect weakens at 236K ($p = 0.228$), likely due to insufficient zoo size (150 models) relative to target complexity.

A.4 Functional vs MSE Autoencoder (109K, SANE)

Table 9: AE loss ablation for SANE (109K). Recon acc = accuracy of decoded weight vectors.

AE Loss	Recon Acc	Zero-shot	Gap vs Kaiming	Better Min?
MSE-only	33%	69%	−0.46pp	No
Functional	97%	81%	+0.41pp	Yes

A.5 Adam vs SGD Optimiser (Letter-Unseen)

Table 10: Optimiser comparison at 50K, letter-unseen, $n = 50$.

Method	+3000 FT	Gap HN–Kaiming	p
HyperNet + SGD	98.33%	+0.51pp	0.00003
HyperNet + Adam	98.46%	+0.18pp	0.032
Kaiming + SGD	97.82%	<i>baseline</i>	
Kaiming + Adam	98.28%	<i>baseline</i>	

The better-minimum effect holds with Adam ($p = 0.032$) but the gap is smaller because adaptive learning rates partially compensate for suboptimal initialisation.